

## CSCE 670 INFORMATION RETRIEVAL · MODULE 08 · LECTURE NOTES

# Vector Search at Scale

*Nearest neighbours when exact is impossible.*

## MODULE OUTCOMES

- Explain why exact search does not scale.
- Explain the graph-based and partition-based approaches.
- Explain quantisation and what it costs.
- Explain the recall-latency-memory trade.
- Choose and tune an index against a stated requirement.

## 1 Why approximate

### EXACT NEAREST-NEIGHBOUR SEARCH IN HIGH DIMENSION IS LINEAR IN THE COLLECTION, WITH NO BETTER OPTION

A brute-force scan computes the distance to every vector — which is a few hundred million floating-point operations for a million 768-dimensional vectors, **per query**, and scales linearly from there.

And the tree structures that work well in low dimension fail here. A **k-d tree degenerates to something close to a full scan above roughly twenty dimensions**, because of **CSCE 676 Module 05 §1's distance concentration**: the pruning bounds stop pruning when all the distances are similar.

So the honest position is that exact search is simply **unavailable at scale** — and **approximate search with a measured recall is the only option rather than a shortcut**, which is **CSCE 676 Module 01 §2's framing** arriving in this course.

Which makes recall the quantity to measure: **what fraction of the true nearest neighbours did your index actually return?** — and it is measurable by brute force on a sample of a few thousand queries, which is cheap and is the exercise nobody runs.

## 2 The two families

**GRAPH-BASED (HNSW)**

build a navigable small-world graph: each vector links to some of its near neighbours, in layers of decreasing sparsity. Search greedily from the top layer down, following the closest link at each step.

Excellent recall at low latency; large memory; slow to build; awkward to delete from.

**PARTITION-BASED (IVF)**

cluster the vectors with k-means, and store an inverted list per centroid. At query time, search only the nearest few lists.

Compact, fast to build, and the recall depends on how many lists you probe -- which is a dial.

**AND THE HASHING FAMILY (LSH)**

CSCE 676 Module 02's construction, applied to vectors. Strong provable guarantees, and beaten in practice by the two above, which have none.

SO: HNSW for quality, IVF for memory and build time, and both are usually combined with section 3.

**The methods with strong guarantees lost in practice to the methods without them — which is worth noticing honestly, and is unusual.** Locality-sensitive hashing has a provable recall bound and HNSW has none, and HNSW is what everybody deploys. **The guarantee-versus-performance observation is an honest one about this field**, and the lesson is that a guarantee is valuable when it is tight and a loose guarantee may be worth less than a measured empirical result (Module 05's measurement discipline applied to algorithms).

**AND THE RECALL DIAL IS WHAT MAKES THESE USABLE**

**Both families expose a parameter that trades recall against latency** — the number of inverted lists probed in IVF, or the size of the search frontier in HNSW — and both are adjustable at query time without rebuilding.

**So you can set the operating point from a requirement** rather than accepting whatever the library default happens to give — **which is the difference between engineering the system and configuring it**, and is Module 08 §4's discipline.

**And the curve is steep at the top: going from 95% to 99% recall frequently costs several times the latency**, which is a **decision worth making deliberately** rather than by accepting a round number.

**Which depends entirely on the stage you are serving: a first-stage retriever feeding a cross-encoder reranker over 500 candidates needs recall at 500, not recall at 10** — and **that is a very much easier target** (Module 06 §4), which means the right operating point is far cheaper than it first appears.

### 3 Quantisation

#### THE PROBLEM

a billion 768-dimensional float32 vectors is about 3 TB. The index structure is a small fraction of that; the VECTORS are the cost.

#### SCALAR QUANTISATION

float32 -> int8. Four times smaller, a small recall loss, and trivial to implement. Do this first, before anything more sophisticated.

#### PRODUCT QUANTISATION

split each vector into m subvectors, cluster each subspace separately, and store m centroid ids. 768 floats -> 96 bytes is routine. Distances are computed from precomputed lookup tables, which is also faster than arithmetic on the originals.

#### BINARY QUANTISATION

one bit per dimension. 32 times smaller, and it requires a reranking pass over the full vectors to recover the lost precision.

AND THE PATTERN: quantise for the search, then rerank the top candidates with exact distances.

**Quantise to search, then rerank exactly** — which is **the same two-stage logic as Module 06 §4, one level down**: use a cheap approximation to narrow the candidate set, then an expensive exact computation to order it. **The recurring two-stage pattern is worth naming explicitly**, because it appears at three levels in this course — retrieve then rerank, quantise then rerank, and chunk then expand (CSCE 638 Module 11 §3) — and recognising it makes each instance easier to reason about.

### 4 Choosing and tuning

- **State the requirement first: recall at k, the latency budget, the memory budget, and the update rate — all four, because they conflict** and because no index is best on all of them.
- **Measure recall by brute force on a sample** of a few thousand queries against the full collection — **which is the only way to actually know, and it is cheap** since it is done once offline.
- **Then: HNSW if memory permits and updates are rare; IVF with product quantisation if memory is tight or the collection grows continuously** — which covers most real decisions.
- **And tune the probe or frontier parameter against your latency budget, reading the recall off the measured curve rather than accepting a default** — the curve takes an afternoon to produce and is reusable.

- **Beware of deletes. Graph indexes handle deletion badly**, usually by marking tombstones and rebuilding periodically — **which is a real operational cost** that does not appear in any benchmark. **The update rate is the requirement most often forgotten**, and **it is the one that rules out the highest-quality index**.

#### AND WHAT THIS COSTS COMPARED TO A LEXICAL INDEX

**Memory, substantially: a dense index is typically very much larger than an inverted index over the same collection**, even after quantisation — because an inverted index stores only the terms that appear and a dense index stores a full vector per document.

**Build time, substantially** — encoding every document through a transformer is far more expensive than tokenising it — and **a full rebuild is required on every embedding model change** (Module 07 §4's commitment).

**And operational complexity: the recall parameter is a quality knob that simply does not exist in a lexical system**, and **a misconfigured one degrades results silently** with no error and no obvious symptom.

**Which is worth stating because the comparison between lexical and dense retrieval is usually made on quality alone** — and **the right comparison includes the memory, the rebuild cost, and the additional thing that can be set wrong**, all of which favour the lexical index and none of which appear in a benchmark table.

## Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

| Resource                                            | What it covers for this module                                                                                |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Malkov & Yashunin — HNSW (free)                     | <b>§2's first family in the original</b> — the layered navigable graph, and why greedy search on it works.    |
| Jégou, Douze & Schmid — Product Quantization (free) | <b>§3's main technique</b> — the subspace decomposition and the lookup-table distance computation.            |
| The FAISS documentation and wiki (free)             | <b>§4 practically</b> — the index type guidance, which is the best available decision tree for this.          |
| ANN-Benchmarks (free)                               | <b>§2 and §4 measured</b> — recall against latency across methods and datasets, which is how to compare them. |

## Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Time a brute-force scan** over a million vectors and extrapolate to a billion.
- 2 **Build a k-d tree** and measure its pruning effectiveness at 5, 20, and 100 dimensions.
- 3 **Measure recall by brute force** for an approximate index on a thousand queries.
- 4 **Build HNSW and IVF** on the same vectors and compare memory, build time, and the recall-latency curve.
- 5 **Plot the recall-latency curve** and find the cost of going from 95% to 99%.
- 6 **Measure recall at 10 and at 500** for the same index, and explain the difference.
- 7 **Apply scalar quantisation** and measure the memory saving and recall loss.
- 8 **Apply product quantisation** at two compression levels and compare.
- 9 **Add an exact reranking pass** over the top 100 and measure the recall recovery.
- 10 **Delete 10% of your HNSW index** and report what happens.

## Self-check

---

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is exact high-dimensional search linear, and why do trees fail?
- 2 What quantity should you measure, and how?
- 3 Describe HNSW and IVF, and the trade between them.
- 4 What is unusual about which family won?
- 5 What does the recall dial let you do, and why is the curve steep at the top?
- 6 Why does the stage determine the recall target?
- 7 Why are the vectors rather than the index the memory problem?
- 8 Name three quantisation methods and the pattern they share.
- 9 Give the four requirements to state, and which is most often forgotten?
- 10 Name three costs of a dense index over a lexical one.